AI speed. Human accountability.

CREDSHIELDS · SMART CONTRACT AUDIT

Smart Contract Audit

Dust

Confidential - Prepared for the Dust Protocol Engineering Team

This report presents the findings of a smart contract security audit conducted against Dust Smart Contracts. Findings were identified through manual review of the contract source by the CredShields' security team.

0	1	4	5	2	12
CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL FINDINGS

CredShields Technologies Pte. Ltd. · Singapore
credshields.com · info@credshields.com

CONFIDENTIAL

/ CONTENTS

Table of Contents

01 Executive Summary

02 Vulnerability Summary

03 Detailed Findings

H001 `DustRouter` keeps the unspent half of a partial fill, and any unprivileged address can take it — HIGH

M001 Sweep fee is charged on the gross amount and re-charged on returned leftover, letting the effective fee exceed the immutable `MAX_FEE_BPS` cap — MEDIUM

M002 `buyLeg` credits the pool-reported token output instead of the vault's realized balance delta — MEDIUM

M003 A user cannot revoke the sweeper's standing authority over their accrued stable, and the only price bound is the keeper's own calldata — MEDIUM

M004 `sweep` pins only the basket composition: the accrued amount, execution venue, fee basis, treasury and minimum are all mutable between quote and execution — MEDIUM

L001 Sweep settlement transfers are not leg-isolated, so a frozen treasury or sweeper bricks sweeping for all users — LOW

L002 `buyLeg` never bounds `spent` against the leg's slice, so a non-canonical listed pool can drain a swept user's other-leg funds — LOW

L003 Use `safeTransfer` / `safeTransferFrom` instead of `transfer` / `transferFrom` — LOW

L004 `DustCapture` enforces its TWAP bound per callback rather than per swap, and credits the vault from the pool's self-reported delta — LOW

L005 Outdated and unpinned compiler pragma — LOW

I001 Repeated partial sweeps drift a user's realized allocation away from their chosen basket weights — INFO

I002 `minOuts[i] == 0` is rejected before the delisted-leg skip, reverting an otherwise-skippable sweep — INFO

– Lead Reviewers & Signatories

Executive Summary

CredShields conducted a security audit of the Dust Smart Contracts. The assessment identified a total of 12 findings, categorized as 0 Critical, 1 High, 4 Medium, 5 Low, and 2 Informational.

The most significant finding, classified as **High**, is [H001] related to the DustRouter. It was observed that the DustRouter retains the unspent portion of a partial fill, which can then be claimed by any unprivileged address. This vulnerability presents a direct financial risk and should be prioritized for immediate remediation.

Additionally, the audit uncovered four Medium severity issues, including improper fee accounting, unchecked token transfer amounts, missing user revocation, and unpinned execution parameters. Five Low severity findings and two Informational findings were also identified. Post-audit, 11 out of the 12 findings have been fixed, with the remaining one acknowledged by the Dust team.

Audited scope

https://github.com/Dust-Foundation/contracts/tree/01b014f0aae3c0a0265f31a0cfce799e34bcf33c

Retested scope

https://github.com/Dust-Foundation/contracts/tree/2d5c289225ce32b3dac9731707d1bb6858fc44f7

Assessment overview

This security assessment involved a comprehensive smart contract audit for the Dust Smart Contracts. The engagement focused on identifying potential vulnerabilities, design flaws, and adherence to security best practices within the codebase. Our methodology combines a meticulous manual review by experienced security researchers with automated analysis tools to detect common attack vectors, reentrancy issues, access control weaknesses, financial calculation errors, and gas optimization opportunities. This multi-faceted approach aims to ensure the robustness and integrity of the smart contracts against a wide range of threats.

The audit identified a total of twelve findings, comprising one High, four Medium, five Low, and two Informational severity issues. The project team has demonstrated a strong commitment to security, with an impressive retest status indicating that eleven out of the twelve identified issues have been fixed, and one has been acknowledged.

The single High severity finding, **Unhandled Partial Fill / Unbounded Callback Payment**, highlighted a critical issue where the DustRouter could retain the unspent half of a partial fill, making these funds accessible to any unprivileged address. This vulnerability posed a direct risk of unauthorized fund access.

Medium severity findings primarily centered on financial integrity and user control mechanisms. **Improper Fee Accounting / Financial Math Error** revealed a scenario where the sweep fee could be charged on the gross amount and re-charged on leftovers, potentially exceeding the intended maximum fee cap. **Unchecked Token Transfer Amount / Ledger-Custody Divergence** noted that `buyLeg` credited the pool-reported token output instead of the vault's actual realized balance delta, creating a potential discrepancy. Furthermore, two findings, **Missing User Revocation / Insufficient Slippage Control in Automated Swaps** and **Unpinned Execution Parameters / Missing Slippage Protection in Automated Swaps**, pointed to deficiencies in user protections for automated swaps, including a lack of user revocation for sweeper authority and mutable execution parameters between quote and execution.

Low severity findings addressed potential denial-of-service risks, unexpected behaviors, and best practices. **Transfers to Addresses That Cannot Receive Funds / External-Call DoS** indicated that a frozen treasury or sweeper could halt sweeping for all users. **Insecure Use of External Calls / Unbounded Callback Payment** identified a risk where a non-canonical listed pool could drain a swept user's other-leg funds. Recommendations included adopting **Missing Best Practices / Non-Standard ERC-20 Compatibility** by using `safeTransfer` / `safeTransferFrom` for better ERC-20 compatibility, and addressing **Unbounded Callback Payment / Missing Cumulative Bound** related to TWAP bound enforcement and self-reported deltas. An **Outdated Compiler Version** was also noted. Informational findings provided insights into edge cases, such as **Failure to Handle Edge Cases** regarding repeated partial sweeps and the premature rejection of `minOuts[i] == 0`. The high rate of remediation post-audit significantly enhances the overall security posture of the Dust Smart Contracts.

ENGAGEMENT DETAILS

CLIENT	Dust
ASSESSMENT TYPE	Smart Contract Security Audit
AUDIT	Dust Smart Contracts
REPORT DATE	11 Sept 2026
START DATE	10 Sept 2026
END DATE	11 Sept 2026
RETEST DATE	12 Sept 2026
TOTAL FINDINGS	12
REVIEWER	Shashank (shashank@credshields.com) · CredShields

/ FINDING SEVERITY DISTRIBUTION

0	1	4	5	2
CRITICAL	HIGH	MEDIUM	LOW	INFO



/ 02: AT A GLANCE

Vulnerability Summary

Every finding from this assessment, ordered by severity. Follow an ID for the full write-up.

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
H001	`DustRouter` keeps the unspent half of a partial fill, and any unprivileged address can take it	HIGH	Unhandled Partial Fill / Unbounded Callback Payment (SCWE-083, SCWE-042)	✓ FIXED
M001	Sweep fee is charged on the gross amount and re-charged on returned leftover, letting the effective fee exceed the immutable `MAX_FEE_BPS` cap	MEDIUM	Improper Fee Accounting / Financial Math Error (SCWE-124)	✓ FIXED
M002	`buyLeg` credits the pool-reported token output instead of the vault's realized balance delta	MEDIUM	Unchecked Token Transfer Amount / Ledger-Custody Divergence (SCWE-110)	✓ FIXED
M003	A user cannot revoke the sweeper's standing authority over their accrued stable, and the only price bound is the keeper's own calldata	MEDIUM	Missing User Revocation / Insufficient Slippage Control in Automated Swaps (SCWE-090)	✓ FIXED
M004	`sweep` pins only the basket composition: the accrued amount, execution venue, fee basis, treasury and minimum are all mutable between quote and execution	MEDIUM	Unpinned Execution Parameters / Missing Slippage Protection in Automated Swaps (SCWE-090)	✓ FIXED
L001	Sweep settlement transfers are not leg-isolated, so a frozen treasury or sweeper bricks sweeping for all users	LOW	Transfers to Addresses That Cannot Receive Funds / External-Call DoS (SCWE-149)	✓ FIXED
L002	`buyLeg` never bounds `spent` against the leg's slice, so a non-canonical listed pool can drain a swept user's other-leg funds	LOW	Insecure Use of External Calls / Unbounded Callback Payment (SCWE-042)	✓ FIXED
L003	Use `safeTransfer` / `safeTransferFrom` instead of `transfer` / `transferFrom`	LOW	Missing Best Practices / Non-Standard ERC-20 Compatibility (SCWE-110)	✓ FIXED
L004	`DustCapture` enforces its TWAP bound per callback rather than per swap, and credits the vault from the pool's self-reported delta	LOW	Unbounded Callback Payment / Missing Cumulative Bound (SCWE-042)	✓ FIXED
L005	Outdated and unpinned compiler pragma	LOW	Outdated Compiler Version (SCWE-061)	✓ FIXED
I001	Repeated partial sweeps drift a user's realized allocation away from their chosen basket weights	INFO	Failure to Handle Edge Cases (SCWE-083)	ACKNOWLEDGED
I002	`minOuts[i] == 0` is rejected before the delisted-leg skip, reverting an otherwise-skippable sweep	INFO	Failure to Handle Edge Cases (SCWE-083)	✓ FIXED

/ 03: DETAILED FINDINGS

12 findings across the assessment.

Findings are ordered by severity.

H001 · UNHANDLED PARTIAL FILL / UNBOUNDED CALLBACK PAYMENT (SCWE-083, SCWE-042)

HIGH

✓ FIXED

`DustRouter` keeps the unspent half of a partial fill, and any unprivileged address can take it

DESCRIPTION

`DustRouter.swapWithRoundUp` pulls the caller's full `amountIn` into the router at `:82`, then calls `IV3Pool.swap` with `amountSpecified = int256(amountIn)` and the price limit pinned to the extreme at `:99`. A Uniswap v3 exact-input swap against that limit terminates on liquidity exhaustion rather than on `amountSpecified`, so a thin pool consumes less than `amountIn`. `_payPool` at `:115` pays the pool only what the pool asked for. Back in `swapWithRoundUp`, `:104` reads only the token side of the returned deltas; the stable side is never compared to `amountIn`, there is no leftover computation and no refund. The difference stays in the router permanently: the contract declares no owner, no rescue and no admin function, and its own NatSpec at `:8` asserts the opposite property, *"Holds no funds between txs"*.

A non-zero `minOut` does not protect the victim. `:105` compares the *token* side against `minOut`, so it bounds the price of what was bought and never the fraction that was bought. A fill that is honestly priced but a fraction of the size requested clears the check. The event at `:107` then reports `stableIn = amountIn`, so the user's own log states they spent the full amount.

The residue is not merely stuck; it is a pot any address can take. The only code path that moves `DustRouter`'s stable is `_payPool`, which pays a pool the *next* caller names, and `swapWithRoundUp` accepts `pool` as an unvalidated caller parameter: no allowlist, no factory check, and no verification that the stable is even one of the pool's two tokens. An attacker deploys a throwaway contract answering its own pairing check at `:91-92`, calls `swapWithRoundUp` with a nominal `amountIn` and `minOut = 0`, and has its callback claim the router's entire balance. `expectedPool` is cleared only after `swap` returns (`:102`), so one `swap()` may drive the callback repeatedly.

This was measured against live mainnet state, not modelled. Driving the real, factory-canonical, initialized TSLA/USDG 0.05% pool (liquidity 3.87e16) exactly as the router does (exact input, price limit pinned to the extreme), an `amountIn` of 100,000 USDG was consumed only to the extent of 13,873.30 USDG, stranding **86,126.70 USDG, or 86.1% of the input**. The identical call against the deeper TSLA/USDG 0.3% pool (liquidity 1.78e18) absorbed the full 100,000 USDG with nothing stranded, confirming the cause is pool depth rather than a defective venue. The swap succeeds, returning 35.34 TSLA, a fair price for the 13,873 USDG actually consumed, so a tight price-based `minOut` passes. An independent measurement at a different fork block reproduced this at 85.8% stranded with the full attacker leg completing, the attacker's profit equalling the victim's stranded stable exactly.

The sibling path in the same repository does all of this correctly and documents it: `DustSweeper` measures real spend as a balance delta at `:297` / `:303`, computes `leftover` at `:281`, returns it via `DustVault.reaccrue` at `:284`, and rejects both `minOut == 0` (`:252`) and `tokensOut == 0` (`:300`). `DustVault.sol:92-96` names this the *"never strand user stable"* property. Against the same dead pool the sweeper's leg reverts `ZeroOutput` and the slice is returned; the router has no zero-output guard at all and keeps the money.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L82
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L95
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L99
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L104
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L105
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L107

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L110
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L115
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L303

BUSINESS IMPACT

Direct loss of user principal, followed by permissionless theft of it. A single ordinary swap against a legitimate, factory-canonical, live pool strands 86% of the caller's input in an admin-less contract, where any address can remove it for roughly 1 USDG of gas in the same block. No privilege is required at any step: the victim needs no unusual parameters beyond a price-based `minOut` that the mechanism cannot catch, and the attacker needs no role, no listing and no owner action. The exposure at any instant is bounded by the stable resting in the router rather than by protocol TVL, but the measurement shows one transaction can place most of a user's trade there. `DustRouter` has no owner, pause or rescue with which to respond.

RECOMMENDED NEXT STEP

Measure the stable actually consumed as a balance delta across the `IV3Pool.swap` call, exactly as `DustSweeper.buyLeg` does at `:297` / `:303`, and return `amountIn` minus that figure to `msg.sender` before emitting. Bound the callback payment against the committed amount: require `owed <= amountIn` in `_payPool` and decrement the remaining allowance so repeated callbacks cannot exceed it in aggregate. Add the two guards `DustSweeper` already has and `DustRouter` lacks: reject `minOut == 0` (compare `:252`) and reject `tokensOut == 0` (compare `:300`). Reject pools whose pair does not include the stable, and consider restricting `pool` to the sweeper's curated menu or to pools the canonical factory acknowledges, mirroring `script/Deploy.s.sol:283-287`. Finally, recompute the round-up from the spent amount rather than the requested amount, and emit the spent amount at `:107` rather than `amountIn`.

RETEST

This issue is fixed in commit 2d5c289.

M001 · IMPROPER FEE ACCOUNTING / FINANCIAL MATH ERROR (SCWE-124)

MEDIUM

FIXED

Sweep fee is charged on the gross amount and re-charged on returned leftover, letting the effective fee exceed the immutable `MAX_FEE_BPS` cap

DESCRIPTION

`DustSweeper.sweep` computes the treasury fee on the gross debited balance at `:242` (`fee = amount * feeBps / 10_000`), before any leg executes, then splits `investable = amount - fee` across the basket and transfers the fee in full at `:278` whenever `invested > 0` .

The contract states the opposite principle and implements it at exactly one point. Its own comment at `:239-241` reads: *"if the sweep ends up investing nothing … no round-up was actually swept, so no fee is charged and the user gets everything back."* That rule is honoured only by the `invested == 0` branch at `:271` . Since `invested` accumulates `spent` at `:262` , one wei of fill on one leg of twenty makes the branch false and levies the entire gross fee. Everywhere between one wei and full investment, the fee is charged on stable that was handed straight back to the user unspent.

The single-sweep over-charge alone breaks the advertised ceiling, with no recycling required. On a 20.000000 USDG sweep at the launch `feeBps` of 100 across twenty equal legs, `fee` is 0.200000 and each `legIn` is 0.990000; if one leg fills, the fee levied against the stable actually swept is **2,020 bps, four times the `MAX_FEE_BPS` of 500** that `:34-36` advertises as the owner's structural ceiling.

The over-charge then compounds. The uninvested remainder returns through `:281-284` with no fee refund and no marker that it has been taxed, so the next sweep charges the gross fee on the same USDG again, and nothing rate-limits how often `sweep` may run (`minSweepAmount` is `0` on the live deployment). `MAX_FEE_BPS` bounds only the nominal per-sweep rate: neither the per-sweep rate against stable actually invested, nor the cumulative lifetime rate on a unit of round-up.

The precondition is ordinary, and the keeper can also manufacture it. Skipped legs are a first-class outcome of the protocol's own leg-isolation design: a delisted token skips at `:255-258` , and any reverting leg is caught at `:263` and emitted as `LegSkipped` . Separately, the keeper chooses every `minOut` and `:233` validates only the array's *length*, never its values, so setting `minOuts[i] = type(uint256).max` for nineteen of twenty legs makes each revert `SlippageExceeded` at `:301` and be caught, driving the ratio deliberately.

Confirmed by proof-of-concept and re-derived independently as the geometric series `f / (1 − s(1 − f))` for fee rate `f` and skipped weight `s` : with `feeBps = 500` and one 90%-weight leg delisted, repeated sweeps of the recycled leftover routed **34.47% (344.8 USDG) of a 1,000 USDG position** to the treasury, roughly seven times the advertised 5% ceiling. At the launch fee of 1% with `minSweepAmount = 0` , a basket 90% halted still loses 9.17% over its lifetime (9.2× the advertised rate), rising to 16.81% at 95% halted and 50.25% at 99%.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L239-L241
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L242
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L262
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L271
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L278
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L281-L284
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L233
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L34-L36

BUSINESS IMPACT

Any swept user whose basket contains a delisted, halted or thin-pool leg is over-charged, with the excess routed to the owner-controlled treasury; a delisted equity silently taxes every holder of it on each subsequent sweep. The over-charge does not require repetition or malice: a single sweep in which one of twenty legs fills already levies 2,020 bps against stable actually swept, four

times the code-enforced ceiling. Repetition compounds it, and because the keeper both chooses every `minOut` and faces no validation of those values, an owner/keeper (a single key by default at launch, with `owner == treasury` on the live deployment) can force skips and re-sweep to extract up to ~100% of a user's accrued. No protocol insolvency results; the loss is a user-to-treasury value leak. Likelihood of the honest-operation variant is high: skipped legs are designed behaviour, the launch fee is already live at 100 bps, and `minSweepAmount = 0` means nothing halts the recycling.

RECOMMENDED NEXT STEP

Levy the fee on the stable actually invested, not on the gross debit. After the leg loop, compute `fee = invested * feeBps / 10_000`, transfer that to the treasury, and return `amount - invested - fee` to the user via `reaccrue`. Returned leftover then carries no fee and is never re-taxed, so the lifetime effective fee can never exceed `feeBps`.
Reference: https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-124/

RETEST

This issue has been fixed. The fee is now levied on the stable actually invested, with the untaxed remainder re-accrued, so returned stable is never re-charged.

M002 · UNCHECKED TOKEN TRANSFER AMOUNT / LEDGER-CUSTODY DIVERGENCE (SCWE-110)

MEDIUM

✓ FIXED

`buyLeg` credits the pool-reported token output instead of the vault's realized balance delta

DESCRIPTION

`DustSweeper.buyLeg` transfers the swap output `got` to the vault and then calls `vault.creditTokens(user, token, got, spent)`, crediting the user's `tokenBalance` with the pool-reported `got` rather than the amount the vault actually received. The stable side of the same function was deliberately hardened to measure a balance delta (`spent`), but the token side was not. If a listed Stock Token ever charges a transfer fee or rebases downward, the vault receives fewer than `got` tokens while the ledger records `got`, so `Σ tokenBalance[·][token]` exceeds `token.balanceOf(vault)` and the last withdrawer of that token cannot be paid. Every asset the live protocol touches was measured on a mainnet fork (USDG, WETH and all 15 listed Stock Tokens), and all 17 transfer exactly 1:1, so no current token triggers this. Two qualifications keep it material. First, a *uniform* fee-on-transfer is caught by the `balanceOf` floor in `_buy` and merely skips the leg; the mechanisms that actually reach the mis-credit are a downward rebase after crediting, or a fee that exempts pools but applies to the sweeper-to-vault hop, neither of which the floor detects. Second, the blast radius is not one token: TSLA, NVDA, AAPL and SPY all read the same EIP-1967 beacon `0xe10b6f6b…1b00` (implementation `0xb35490d6…5aE2`), so a single beacon upgrade changes transfer semantics for every listed Stock Token at once. The protocol performs three balance-delta measurements in total, all in `DustSweeper` (`:297`, `:303`, `:325`); the other fourteen `transfer` / `transferFrom` calls across the four contracts trust the nominal amount.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L306

BUSINESS IMPACT

Credited token balances outrun the vault's real holdings and the shortfall is pushed silently onto whoever withdraws that token last, an uninvolved user who took no action and has no signal that their ledger entry is unbacked. `DustVault` has no pause, no rescue and no reconciliation, and is wired once via `setModules`, so the insolvency is permanent and unrecoverable without redeploying and migrating. Rated Medium on impact rather than likelihood: the impact is a permanent, unrecoverable per-token insolvency borne by a third party, while the likelihood is low: no live asset exhibits the behaviour today (measured, see Appendix) and the trigger is a shared-beacon upgrade outside the client's control. The finding is forward-looking by construction: the Dust contracts are immutable and cannot react to a token upgrade after listing, so the guard has to exist before it is needed.

RECOMMENDED NEXT STEP

Credit the vault's measured balance delta rather than `got`: read `token.balanceOf(vault)` before and after the transfer and pass the difference to `creditTokens`, mirroring the balance-delta discipline already used for `spent`. This also closes the donation-masking variant.
Reference: https://scs.owasp.org/SCWE/SCSVS-COMP/SCWE-110/

RETEST

This issue is fixed. 'buyLeg' credits the vault's measured balance delta and reverts the leg on a zero delta.

M003 · MISSING USER REVOCATION / INSUFFICIENT SLIPPAGE CONTROL IN AUTOMATED SWAPS (SCWE-090)

MEDIUM

✓ FIXED

A user cannot revoke the sweeper's standing authority over their accrued stable, and the only price bound is the keeper's own calldata

DESCRIPTION

The two keeper-facing surfaces disagree about whether a user may withdraw the keeper's authority, and it is the same key on both sides. `DustCapture` holds no keeper of its own and reads `sweeper.keeper()` live at `DustCapture.sol:162` and `:257`. `DustCapture` gives the user an unconditional kill switch: `setWeeklyCap(0)` at `:119-124`, after which `_consumeCap` refuses every funding source at `:246`. Its NatSpec calls it exactly that at `:118` (*"Opt in / change the cap / opt out (cap 0)"*), and the contract header at `:17-20` states the keeper *"can never do more than"* pull observed round-ups into the user's own position.

`DustSweeper` gives nothing equivalent. `sweep` admits `msg.sender == keeper` for any user at `:227` with no per-user consent term, and `:237` calls `vault.debitAccrued` for the user's entire accrued balance, which `DustVault.debitAccrued` at `:74-81` grants the sweeper unconditionally, with no partial and no ceiling. The only per-user precondition is a non-empty basket at `:231`, and it cannot be un-armed: `_basket` is written at exactly two lines, `:152` and `:162`, both inside `setBasket`, and `:151` rejects `n == 0`, so a user who has ever set a basket has no on-chain way to unset it. Nothing rate-limits how often `sweep` may be called: the file contains no `block.timestamp`, no `block.number` and no per-user counter of any kind, and `:232` compares `basketNonce` without consuming it, so no quote ever expires.

The price bound is one wei, and the keeper sets it. `minOuts` is the caller's own calldata; `:233` validates only its length, `:252` requires only that each entry be non-zero, and `sqrtPriceLimitX96` at `:319` is pinned to the absolute extreme so the pool enforces nothing either. `DustSweeper` contains no oracle, no `observe()`, no `slot0()` and no reference price of any kind. The NatSpec at `:15-16` nevertheless states *"Price is bounded by a caller-supplied nonzero `minOut`"*, a bound the keeper imposes on itself. A keeper sweeping at `minOut = 1` around its own price movement is byte-indistinguishable from an honest sweep except by after-the-fact price analysis, and requires no state change, no listing and no owner action.

The user's only counter is to race the keeper with `DustVault.withdrawAccrued` at `:105`, and that defence is thinner than it appears: `:236` does revert `BelowMinimum` when the balance is zero, but `minSweepAmount` is 0 on the live deployment, so any non-zero balance is immediately sweepable and round-ups accrue automatically from ordinary activity. Staying safe would require withdrawing continuously.

Confirmed by proof-of-concept `test_A5_basketCannotBeRevoked_keeperTakesEverything`.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L227
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L237
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L151-L152
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L162
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L232
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L233
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L252
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L319
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L15-L16
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustVault.sol#L74-L81
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L119-L124
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L246

BUSINESS IMPACT

A malicious or compromised keeper can convert any basket-holding user's entire accrued balance into near-worthless output in a single transaction, repeat it without limit, and the user cannot revoke the authority that permits it. Loss is bounded per user by their accrued round-ups, which are individually small, but applies to every user who has ever called `setBasket` and cannot be opted out of. Beyond the direct exposure, the finding breaks two stated invariants: the header's *"Price is bounded by a caller-supplied nonzero `minOut`"* describes the keeper bounding itself, and `DustCapture`'s *"can never do more than that"* does not hold for the same key acting through `DustSweeper`. Likelihood is low: it requires the keeper key (a separate EOA from the owner on the live deployment, though `setKeeper` lets the owner repoint it), but the bar is materially lower than the report's owner-gated findings, since it needs only calldata rather than an auditable on-chain setup step.

RECOMMENDED NEXT STEP

Give `DustSweeper` the revocation `DustCapture` already has: a `revokeBasket()` that empties `_basket`, or a per-user `sweepEnabled` flag defaulting to off, mirroring `setWeeklyCap(0)` so the same key faces the same user veto on both surfaces. Consume `basketNonce` on each sweep so a quote cannot be replayed indefinitely. Replace the self-imposed price floor with one the user pre-authorises: either a maximum-slippage parameter stored per user and applied to a reference price, or an on-chain TWAP check against the leg's pool in the manner of `DustCapture.maxWethInFor`, so that `minOut` bounds the keeper rather than expressing its intent. Consider also a per-user minimum interval between sweeps, since none exists today.

RETEST

This vulnerability has been fixed in commit 2d5c289.

M004 · UNPINNED EXECUTION PARAMETERS / MISSING SLIPPAGE PROTECTION IN AUTOMATED SWAPS (SCWE-090)

MEDIUM

✓ FIXED

`sweep` pins only the basket composition: the accrued amount, execution venue, fee basis, treasury and minimum are all mutable between quote and execution

DESCRIPTION

A sweep's per-leg `minOuts` are priced off-chain against a state snapshot, then submitted. `sweep` has exactly one staleness guard for that snapshot, `expectedNonce` at `:232`, compared against `basketNonce`, whose sole writer is the user's own `setBasket` at `:165`, and it covers the basket composition alone. Five further inputs decide what is actually bought, and none is pinned: the accrued amount, read fresh at `:235`; the execution venue, resolved late at `:254` from the mutable menu rather than at `setBasket` time; `feeBps` and `treasury`, both read live at `:242`; and `minSweepAmount` at `:236`.

Each moves independently of the quote. A balance that grows between quote and inclusion (one `DustRouter.swapWithRoundUp` at `granularity = 100, multiplier = 10` accrues ~1,000 USDG and does not bump `basketNonce`) enlarges every `legIn` while the fixed `minOut[i]` stays constant, and with `sqrtPriceLimitX96` pinned to the extreme the pool imposes no bound either, leaving the leg open to a sandwich. The owner may `delistStock(T)` then `listStock(T, otherPool)` in one transaction (`:135-136` zero `poolFor` and reopen the `AlreadyListed` gate at `:111`), re-pointing every live basket holding `T` at a venue the user never chose, with no nonce movement and no user-visible signal, since `StockListed` / `StockDelisted` are indexed by token only. And `setFeeBps(0)`, or equivalently `setTreasury(0)` since `:242` tests `treasury != address(0)`, landing between quote and sweep alters `investable` at `:243` and therefore every `legIn` at `:249` while `minOuts[i]` remains an absolute token amount checked at `:301`: **+5.26% input against an unchanged floor** when quoted at the 500 bps maximum, +1.01% at the live 100 bps, with nothing reverting.

Three documented claims do not survive this. `:15-16` states *"same accrued amount + same user basket -> same buys, every time"*, yet both named inputs can be unchanged while the buy differs; `:13-14` states *"The owner cannot change what a user's money buys"*, conceded at `:122-124` for the delist case but not for the re-list; and `:34-36` presents `MAX_FEE_BPS = 500` as structural although it binds the fee line at `:201` and no other quantity, a claim `AUDIT-SCOPE.md:59-61` generalises to *"Owner powers on DustSweeper are enumerated and bounded"*. The powers also co-locate: the constructor sets `owner` and `keeper` to one address at `:100-101`, `script/Deploy.s.sol:210` only *starts* the handover, and on the live deployment `owner == treasury == 0xc1a4b142...5d2b` with `pendingOwner == address(0)`, the separate keeper one `setKeeper` call away. Extraction magnitude once unpinned is covered by M-1, M-3 and L-2 and is not restated here.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L232
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L235
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L254
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L135-L136
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L111
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L242-L243
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L249
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L301
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L236
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L15-L16
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L13-L14

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L34-L36
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L201
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/AUDIT-SCOPE.md#L59-L61

BUSINESS IMPACT

A sweep can execute against materially different economics than the one the keeper priced, with every individual check satisfied and nothing reverting. Two distinct paths: the accrued-amount drift is coincidental and costs the user MEV on the incremental accrual, while the venue, fee-basis and treasury changes are deliberate owner actions available at any moment, requiring no listing of a hostile pool and no unusual calldata. The per-sweep economic effect of the fee manoeuvre is small (1.01% at the live rate, 5.26% at the maximum), but the venue substitution is not bounded at all, since it replaces the pool the user's money trades in while their basket, their nonce and their events all read unchanged. Likelihood is low for the owner-driven variants (they need the owner key, a single EOA with no timelock and no multisig handover in progress) and low-to-moderate for the accrual drift, which needs only ordinary user activity plus a searcher.

RECOMMENDED NEXT STEP

Extend the staleness guard to cover every input the quote depended on. Add an `expectedAmount` (or `maxAmount`) parameter and revert if `vault.accruedStable(user)` exceeds it. Have the caller pass the expected pool per leg and verify it at `:254`, or maintain a menu-generation counter bumped by both `listStock` and `delistStock` and include it in the value checked at `:232`. Pass and check the expected `feeBps`, so a fee change between quote and execution invalidates the quote rather than silently re-scaling every `legIn`. Consume `basketNonce` on each sweep so a quote cannot be replayed. Most durably, express each `minOut[i]` as a per-stable-unit rate applied to the actual `legIn` rather than an absolute token amount, which makes the floor immune to every input above at once. Separately, complete the ownership handover to a timelocked multisig, which removes the deliberate variants without any code change.

RETEST

This issue has been fixed in commit 2d5c289.

L001 · TRANSFERS TO ADDRESSES THAT CANNOT RECEIVE FUNDS / EXTERNAL-CALL DOS (SCWE-149)

LOW

✓ FIXED

Sweep settlement transfers are not leg-isolated, so a frozen treasury or sweeper bricks sweeping for all users

DESCRIPTION

The prior review's leg-isolation fix wraps each leg buy in `try/catch` so one failing token cannot brick a sweep, but the settlement transfers are not similarly isolated: the fee transfer `stable.transfer(treasury, fee)`, the leftover/refund transfers, and the `stable.transfer(sweeper, amount)` inside `DustVault.debitAccrued` all revert the entire sweep on failure. USDG is a Paxos proxy with per-address freeze. If the treasury address is frozen, every sweep with `invested > 0` reverts for all users (the owner can recover via `setTreasury` or `setFeeBps(0)`). If the immutable sweeper address itself is frozen, `debitAccrued` reverts and sweeping is permanently bricked for everyone, since the sweeper is wired once via `setModules` and cannot be replaced. User principal is never at risk in either case: accrued balances are untouched when the sweep reverts and `withdrawAccrued` keeps working, so this is an availability issue, and the freeze is an issuer compliance action rather than an attacker-triggerable one.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L278
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustVault.sol#L79

BUSINESS IMPACT

A single frozen address (treasury or sweeper) disables the sweep function for the entire user base; the sweeper-freeze variant is unrecoverable without a redeploy. User funds remain withdrawable throughout, so there is no loss of principal. Likelihood is low (issuer/stablecoin action, no attacker path).

RECOMMENDED NEXT STEP

Wrap the fee transfer in the same `try/catch` (or a low-level call with a checked-but-non-reverting result) used for legs, skipping the fee if the treasury cannot receive, so a treasury freeze cannot brick settlement. Consider treating an unavailable treasury as fee = 0 for that sweep.
Reference: https://scs.owasp.org/SCWE/SCSVS-COMM/SCWE-149/

RETEST

This issue has been fixed in commit 2d5c289.

L002 · INSECURE USE OF EXTERNAL CALLS / UNBOUNDED CALLBACK PAYMENT (SCWE-042)

LOW

✓ FIXED

`buyLeg` never bounds `spent` against the leg's slice, so a non-canonical listed pool can drain a swept user's other-leg funds

DESCRIPTION

`DustSweeper.sweep` debits the user's entire accrued balance to the sweeper up front and holds it there while every leg runs. Each leg's `buyLeg` measures the stable it paid as a balance delta (`spent = stableBefore - balanceOf(this)`) but never checks `spent <= legIn`, and `_payPool` pays the pool whatever `owed` it reports, with no upper bound. For a canonical Uniswap v3 pool an exact-input swap can never pull more than `legIn`, so this is inert; but `listStock` validates only the pool's `token0` / `token1` pairing, unlike `Deploy.s.sol`, which also requires each pool to originate from the canonical Uniswap v3 factory (`_checkFactoryPool`). A non-canonical or upgradeable/compromised pool that satisfies the pairing check can therefore be listed, and during its own swap it can report `owed` equal to nearly the whole debited amount, deliver only `minOut` -worth of tokens, and let the remaining legs skip for lack of funds. The sibling `DustCapture._paySwap` bounds the analogous payment to `wethLimit`; the sweeper has no equivalent. Confirmed by proof-of-concept: a hostile listed pool pulled 950 USDG for its own 500 USDG leg slice, delivered 1 wei of token, and the victim was left with 50 USDG recoverable out of 1,000.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L303
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L109

BUSINESS IMPACT

A swept user loses the funds allocated to their other basket legs to the operator of a malicious/non-canonical listed pool, breaking the protocol's central "owner powers touch no user balance" invariant. It requires the `DustSweeper` owner to list a non-canonical pool: `poolFor` has exactly two writers, `listStock` (`:116`) and `delistStock` (`:136`), both `onlyOwner`, so no unprivileged actor can reach this path. Rated Low on that basis: the trigger is a trusted-role action, and the marginal capability over M-1 (which already reaches ~100% extraction by forcing skips and re-sweeping) is that the drain is atomic and per-user rather than incremental and interruptible. The rating is a function of key management rather than of the code: owner, treasury and fee recipient are currently one EOA with `pendingOwner == address(0)`, and handing ownership to a timelocked multisig removes the remaining likelihood without any code change.

RECOMMENDED NEXT STEP

Add `if (spent > legIn) revert SlippageExceeded();` in `buyLeg` immediately after `spent` is computed. This bounds every leg to its own slice and never triggers for an honest exact-input swap. Have `listStock` also validate the pool against the canonical factory exactly as `Deploy.s.sol._checkFactoryPool` already does, so the runtime setter maintains the provenance the deploy path establishes.
Reference: https://scs.owasp.org/SCWE/SCSVS-COMM/SCWE-042/

RETEST

This issue has been fixed in commit 2d5c289.

L003 · MISSING BEST PRACTICES / NON-STANDARD ERC-20 COMPATIBILITY (SCWE-110)

LOW

✓ FIXED

Use `safeTransfer`/`safeTransferFrom` instead of `transfer`/`transferFrom`

DESCRIPTION

All fourteen token movements across the four in-scope contracts call `transfer()` / `transferFrom()` directly through the minimal `IERC20` interface declared in `interfaces/IDust.sol`, rather than OpenZeppelin's `SafeERC20` wrappers. OpenZeppelin discourages the bare forms because `safeTransfer` / `safeTransferFrom` normalise the return-value handling that the ERC-20 standard leaves inconsistent across implementations.

One qualification specific to this codebase: the usual motivation for `SafeERC20`, silently ignored boolean return values, does not apply here. Every one of the fourteen call sites already wraps the call in `if (!...) revert`, reverting with `TransferFailed` or `BadCallback`, and a false-returning token was confirmed to be caught at each layer by proof-of-concept (`test_S79_false_returning_token_is_caught_everywhere`). The codebase is already correct on that axis.

The residual gap is the other half of what `SafeERC20` provides: tolerance of tokens that return no value at all. Because `IDust.sol` declares `transfer` / `transferFrom` as `returns (bool)`, a token following the non-standard no-return convention (USDT being the canonical example) causes the ABI decode to revert. Confirmed by proof-of-concept `test_S80_non_returning_token_reverts_not_silently_succeeds`: the call reverts rather than silently succeeding, so the behaviour fails closed and no value is lost, but such a token cannot be integrated at all. This matters because the asset set is not fixed: `listStock` is owner-extensible at runtime, and every listed Stock Token sits behind a shared upgradeable beacon (see M-2), so the transfer semantics of an already-listed asset can change after deployment while the Dust contracts remain immutable.

`src/ComposabilityProbe.sol` also uses bare `transfer` / `transferFrom`, but is documented as throwaway code never deployed to mainnet and is excluded here.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustVault.sol#L79
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustVault.sol#L109
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustVault.sol#L117
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L82
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L85
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L115
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L169
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L300
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L305
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L272
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L278
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L283
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L305
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L333

BUSINESS IMPACT

No loss of funds and no incorrect accounting in the current deployment: return values are checked everywhere, and every asset the protocol touches today (USDG, WETH and all 15 listed Stock Tokens) was measured transferring exactly 1:1 on a mainnet fork

(see Appendix). The impact is forward-looking compatibility and defence in depth. A stable or Stock Token that adopts the no-return convention, whether newly listed or introduced by a beacon upgrade to an existing one, would revert on every protocol path touching it: sweeps of that leg would skip, and for the stable, accrual and withdrawal would halt outright. Adopting `SafeERC20` also brings `forceApprove` /allowance helpers should an approval pattern be introduced later.

RECOMMENDED NEXT STEP

Use OpenZeppelin's `SafeERC20` library and replace the fourteen bare calls with `safeTransfer` / `safeTransferFrom`. The existing `if (!...) revert` wrappers become redundant and should be removed in the same change, since `SafeERC20` reverts internally; leaving both would be harmless but misleading. Note that this changes the revert signature on failure from the contracts' own `TransferFailed` / `BadCallback` errors to OpenZeppelin's, so any keeper or UI error handling that matches on those selectors needs updating alongside.
Reference: https://docs.openzeppelin.com/contracts/5.x/api/token/erc20#SafeERC20

RETEST

This issue has been fixed in commit 2d5c289.

L004 · UNBOUNDED CALLBACK PAYMENT / MISSING CUMULATIVE BOUND (SCWE-042)

LOW

FIXED

`DustCapture` enforces its TWAP bound per callback rather than per swap, and credits the vault from the pool's self-reported delta

DESCRIPTION

`DustCapture` states an unconditional guarantee at `:29-31`: *"The WETH the keeper may spend per stable unit is bounded on-chain by the pool's own 10-minute TWAP plus `MAX_SLIPPAGE_BPS`, so a compromised keeper cannot overpay."* Both enforcement points are per-event rather than cumulative, so the guarantee holds for a single callback and not for a swap.

`_captureViaSwap` consumes the weekly cap once at `:259`, in stable units for `amountOut` only (it never sees WETH), computes `limit` at `:262-263`, writes the `pending` struct at `:265` and calls `ethPool.swap` at `:266`. Inside the callback, `:296` bounds one invocation. Nothing marks `pending` consumed: `:288` copies it to memory, `_paySwap` never clears or decrements the storage value, and the `delete` at `:273` runs only after `swap()` has returned. A pool that invokes `uniswapV3SwapCallback` N times within one `swap` is therefore paid N times, each payment passing `:293` and `:296` with identical deltas.

The post-swap check cannot recover the difference either. `:277-278` compares the single aggregate the pool returns against `limit`, a report rather than a measurement of what actually left the contract, so a pool presenting one clean aggregate while paying itself N times satisfies it. The cumulative spend is bounded by nothing except the user's ETH float at `:302-303` or their WETH allowance at `:300`, while `configOf.usedThisWeek` is charged once, for `amountOut`. The same absence of a measured check applies to the stable side: the vault is credited `amountOut` on the strength of the reported delta, with no verification that the stable reached `DustVault`.

Confirmed by proof-of-concept: `test_C1_ethFloatDrainedByRepeatedCallback` spends 8× the TWAP bound from a user's ETH float against a single 10 USDG credit; `test_C2_walletWethAllowanceDrainedByRepeatedCallback` spends 20× from a wallet allowance; `test_C3_vaultCreditedForStableThatNeverArrived` credits 250 USDG against a vault holding zero.

This is not reachable on the deployed system, by any actor. `ethPool` is declared `immutable` at `:47`; the constructor is its only writer and no setter exists, so neither the keeper, the owner, a user nor an attacker can substitute a pool. The deployed pool's behaviour was measured rather than assumed: on a mainnet fork of `0x52e65B17...71Ca`, one `swap()` invoked the callback exactly once, paid exactly its reported delta (10161777963115356 wei) and delivered exactly its reported output, with provenance confirmed on-chain (`pool.factory()` returns the canonical factory and `getPool(WETH, USDG, 100)` round-trips). A canonical Uniswap v3 pool does not invoke the callback more than once.

Two facts keep this above informational. The provenance guarantee lives in `script/Deploy.s.sol:191`, not in the contract (`DustCapture`'s own constructor validates only `token0` / `token1` at `:102-107`), so it does not travel with the code. And `_checkFactoryPool` runs on the mainnet path only: `_deployTestnet` takes `ETH_POOL` from an environment variable with no provenance check at all, so any redeployment, fork or non-mainnet instance inherits a constructor weaker than the script that protects production today.

This is the same latch defect as H-1 (`DustRouter`) and L-2 (`DustSweeper`), reached through a third surface. The three differ only in who can supply the pool: anyone, the owner, and in this case nobody.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L288
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L296
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L273
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L277-L278
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L259
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L300
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L302-L303
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L29-L31

BUSINESS IMPACT

None against the deployed instance. The precondition requires a pool that misbehaves, the pool is immutable and factory-verified, and it was measured invoking the callback exactly once. The exposure is to future deployments and forks, where the constructor does not enforce what the mainnet deploy script does. Were the precondition met, the impact would be severe and immediate: a user's entire ETH float or WETH allowance, commonly an infinite approval, spent in one authorised keeper call against a single round-up's credit and a single charge to the weekly cap, plus unbacked vault credit that the pooled-custody ledger converts into a loss for whichever user withdraws last. Rated Low on that basis: the defect is real and the documented guarantee is false as written, but no actor on the live system can reach it.

RECOMMENDED NEXT STEP

Decrement the bound as it is consumed. In `_paySwap`, after computing `owed` and checking it against `p.wethLimit`, write the reduced value back to storage (`pending.wethLimit -= owed`, reverting on underflow) so repeated callbacks cannot exceed the per-swap limit in aggregate. For the credit, measure rather than trust: record `stable.balanceOf(address(vault))` before the swap and credit the realised delta, the technique `DustSweeper.buyLeg` already uses for `spent`. Independently, move the factory-provenance check from `script/Deploy.s.sol:283-287` into the `DustCapture` constructor so the guarantee travels with the contract rather than with one deployment path, and amend the NatSpec at `:29-31` to state the bound it actually enforces.

RETEST

This issue has been fixed in commit 2d5c289.

L005 · OUTDATED COMPILER VERSION (SCWE-061)

Outdated and unpinned compiler pragma

LOW

✓ FIXED

DESCRIPTION

Every in-scope file declares a floating pragma. The four core contracts and the interface use `pragma solidity ^0.8.24`, while the two vendored libraries use `pragma solidity ^0.8.0`. A caret pragma fixes only the lower bound, so the source does not determine which compiler produces the bytecode, and `foundry.toml` pins no `solc` version either, leaving the choice to whatever the toolchain resolves at build time.

That divergence is already live rather than theoretical. Every build artifact in `out/` reports `0.8.35+commit.47b9dedd`, while the source advertises 0.8.24 and the libraries accept anything from 0.8.0 upward. A reviewer reading the pragma, or a third party reproducing the build from source alone, would not necessarily arrive at the compiler that produced the deployed code. The `^0.8.0` range matters most: it predates several codegen fixes, and the two files carrying it are `TickMath` and `FullMath`, which perform the fixed-point arithmetic behind `DustCapture.maxWethInFor`.

Solidity is under active development and point releases carry security patches, codegen bug fixes and optimiser improvements. An open upper bound means a later build can silently pick up a compiler the code was never reviewed or tested against, and an open lower bound means it can equally be built with one that is years old.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustVault.sol#L2
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustRouter.sol#L2
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L2
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustCapture.sol#L2
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/interfaces/IDust.sol#L2
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/libraries/TickMath.sol#L2
- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/libraries/FullMath.sol#L2

BUSINESS IMPACT

No direct exploit path and no user funds at risk, which is why this is rated Low. The practical consequences are reproducibility and review integrity: bytecode cannot be derived from the source alone, a contract audited under one compiler may be deployed under another, and codegen defects fixed after 0.8.0 remain within the accepted range for the two arithmetic libraries. Because the Dust contracts are immutable once deployed, a compiler-introduced defect cannot be patched afterwards, so the compiler choice is a one-time decision that the source should record rather than delegate to the build environment.

RECOMMENDED NEXT STEP

Pin the compiler rather than floating it. Replace the caret pragma with an exact version in all seven in-scope files, `pragma solidity 0.8.35;`, which matches the compiler that produced the current artifacts and raises the two vendored libraries off `^0.8.0` at the same time. Add `solc = "0.8.35"` to `foundry.toml` so the toolchain cannot drift between builds, and re-run the full test suite after the change. If the vendored libraries are kept byte-identical to their upstream Uniswap originals as a deliberate policy, pin them to the same exact version and note the deviation from upstream in a comment.

RETEST

This issue has been fixed in commit 2d5c289.

I001 · FAILURE TO HANDLE EDGE CASES (SCWE-083)

INFO

ACKNOWLEDGED

Repeated partial sweeps drift a user's realized allocation away from their chosen basket weights

DESCRIPTION

When a basket leg is persistently skipped (halted or delisted token), its stable slice is returned to `accruedStable` and, on the next sweep, re-split across the whole basket rather than parked for the intended token. Over repeated keeper-driven sweeps this progressively funnels the halted leg's intended allocation into the user's still-liquid legs, so the realized allocation drifts away from the weights the user set in `setBasket`. There is no direct loss of funds, and the user can interrupt the drift at any time via `withdrawAccrued`, but a keeper timing sweeps can push a user's dust into assets they did not weight toward while their chosen (halted) asset is unavailable.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L254-L266

BUSINESS IMPACT

A user may end up holding a basket meaningfully different from the one they composed when some legs are chronically unbuyable. This is a fairness/UX concern, not a solvency or theft issue.

RECOMMENDED NEXT STEP

Consider parking a skipped leg's slice as a pending allocation for that specific token (or leaving it withdrawable but not re-split) rather than redistributing it across the basket, so a temporary halt does not silently rebalance the user.

RETEST

This issue is acknowledged. Client comment : documented in the DustSweeper NatSpec and in "Known limitations" above. No code change.

I002 · FAILURE TO HANDLE EDGE CASES (SCWE-083)

INFO

`minOuts[i] == 0` is rejected before the delisted-leg skip, reverting an otherwise-skippable sweep

✓ FIXED

DESCRIPTION

In `DustSweeper.sweep` the `if (minOuts[i] == 0) revert ZeroMinOut();` check is evaluated before the `poolFor[leg.token] == 0` delisted-leg skip. If a leg's token has been delisted (so it would otherwise be skipped cleanly), a `minOut` of 0 for that leg still reverts the whole sweep instead of skipping the leg. The keeper controls `minOuts` and can simply pass a non-zero placeholder for delisted legs, so there is no permanent denial of service, but the ordering is surprising and makes the keeper's job more error-prone than necessary.

AFFECTED CODE

- https://github.com/Dust-Foundation/contracts/blob/01b014f0aae3c0a0265f31a0cfce799e34bcf33c/src/DustSweeper.sol#L252

BUSINESS IMPACT

A keeper that passes `minOut = 0` for a delisted leg wastes gas on a reverted sweep. No funds are affected and the keeper can trivially avoid it.

RECOMMENDED NEXT STEP

Evaluate the `poolFor[leg.token] == 0` skip before the `minOuts[i] == 0` check, so a delisted leg is skipped regardless of its supplied `minOut`.

RETEST

This issue has been fixed in commit 2d5c289.

DISCLOSURE

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or issue and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

REVIEWER(S) & CONTACT

Shreyas Koli
Security Auditor II

Prasad Kuri
Security Auditor

Neel Shah
Security Auditor

LEAD REVIEWERS & REPORT SIGNATORIES



Shashank
Co-founder & CEO · CredShields

Security analyst and penetration tester with **12 years of experience** in pentesting and **5 years** in managing bug bounty programs. Listed in **Google, Facebook, Apple,** and 30+ websites' hall of fame for reporting valid security issues.

CVE-2017-7063 CVE-2017-7062 CVE-2017-2458



Aditya Dixit
Audit & Research Lead · CredShields

Senior security researcher and penetration tester with **10+ years** in cybersecurity. Led pentest teams at **Cobalt and HackerOne**; previously at Binary.com. Executed **500+ pentests** across banking, fintech, payments, healthcare, telecom, and media, covering web, mobile (iOS/Android), network, cloud, code review, and smart contracts.

OSCP CREST CPSA AWS Security Specialty AWS Cloud Practitioner Qualys VMDR